

Benchmark Optimization, Analysis, and Hardware Acceleration

pyperformance N-body and Pyflate | Course QEMU measurements
Software correctness, profiling, optimization, and synthesizable RTL

Method and success criteria

Two approved workloads: N-body and Pyflate

Understand algorithms and data structures before changing code

Measure with rigorous pyperf on one pinned QEMU vCPU

Locate hotspots with Linux perf and flame graphs

Require identical output/state and at least 7% improvement

Propose explicit ready/valid hardware interfaces and software fallback

Experimental environment and correctness

Ubuntu 5.15.0-1106-kvm under KVM

1 vCPU: Intel Xeon E5-2630 v3 at 2.40 GHz

CPython 3.10.12; pyperformance 1.14.0; pyperf 2.10.0

N-body: complete positions, velocities, and energy matched

Pyflate: 399,360 output bytes matched byte-for-byte

Pyflate MD5: afa004a630fe072901b1d9628b960974

Both SystemVerilog self-checking simulations passed

N-body: workload and baseline

Five solar-system bodies; ten unordered interaction pairs

20,000 dependent time steps per benchmark operation

Per pair: displacement, inverse distance cubed, two velocity updates

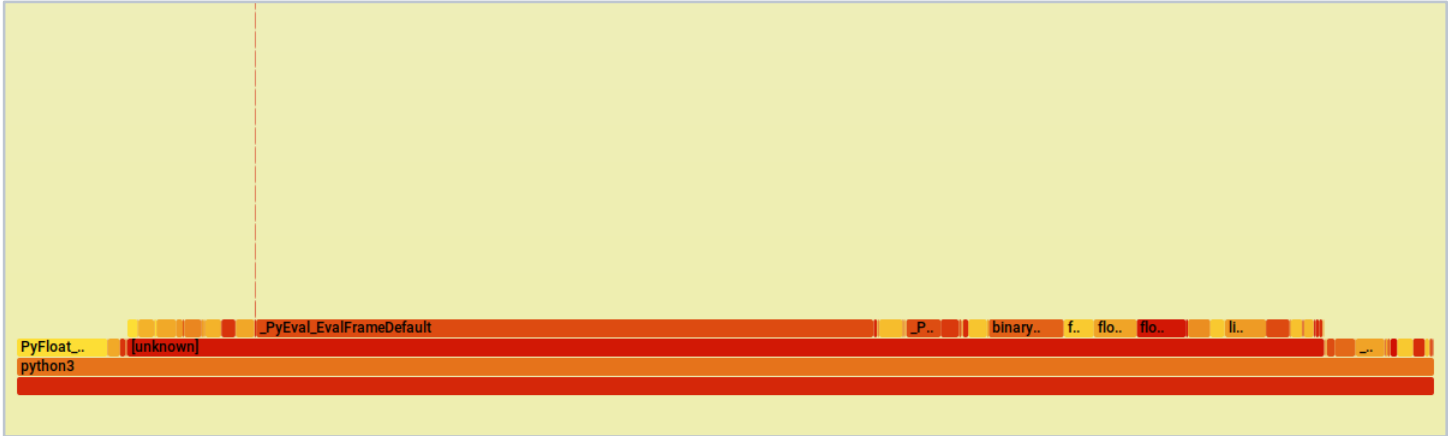
Per step: integrate all five positions

Complexity: $O(T \cdot N^2)$; N is fixed, so CPython overhead dominates

Baseline storage: nested Python lists/tuples and boxed float objects

N-body baseline profile

29,396 CPU-clock samples; zero lost samples



43.54% CPython frame evaluation

7.09% float creation

5.49% generic binary dispatch

3.46% float multiplication

2.88% list assignment

Only 2.03% in inverse-distance pow

N-body software optimization

Store positions, velocities, and masses in dense NumPy float64 arrays

Compile the complete pair and integration loops with Numba nopython

Warm compilation before timing; measure cold start separately

Keep pair order, update order, float64, and fast-math disabled

Return an energy checksum so the computation remains observable

Result: remove interpreter dispatch and expose the true math hotspot

N-body result: 23.13x faster

Baseline



230 ms

Optimized



9.94 ms

95.68% time reduction; required threshold: 7%

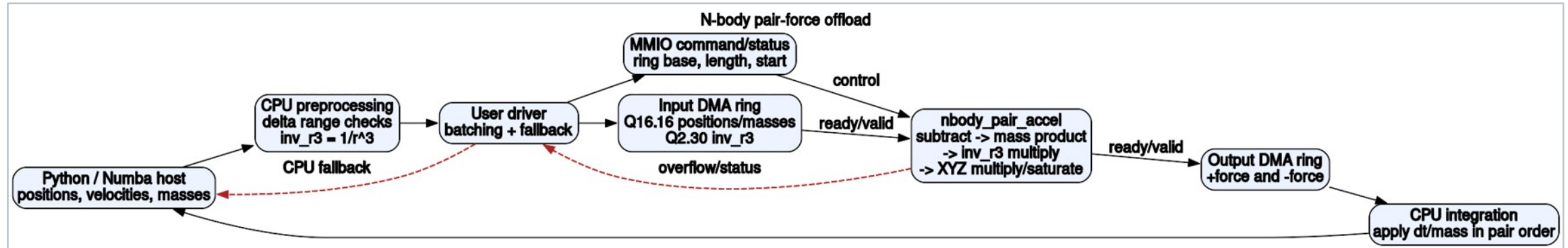
120 values each; CV 1.60% baseline and 2.52% optimized

Optimized profile: CPython evaluation falls from 43.54% to 0.01%

Power-related functions become about 70.25% of execution

Cold import + compilation: 1.288 s; break-even near six operations

N-body hardware proposal and limits



Q16.16 pair-force datapath; Q2.30 host-provided inverse distance cubed

One-cycle ready/valid reference; equal-and-opposite saturated outputs

Standalone PCIe offload is not viable for only ten dependent pairs/step

Production design must add reciprocal square root, accumulation, and multi-step residency

At $P=70.25\%$ and $S=20$, ideal Amdahl bound is about 3.0x before transfer cost

Pyflate: workload and baseline

Pure-Python BZip2 decode of interpreter.tar.bz2

Bit extraction -> canonical Huffman -> MTF/RLE -> inverse BWT

Baseline reads one byte per method call

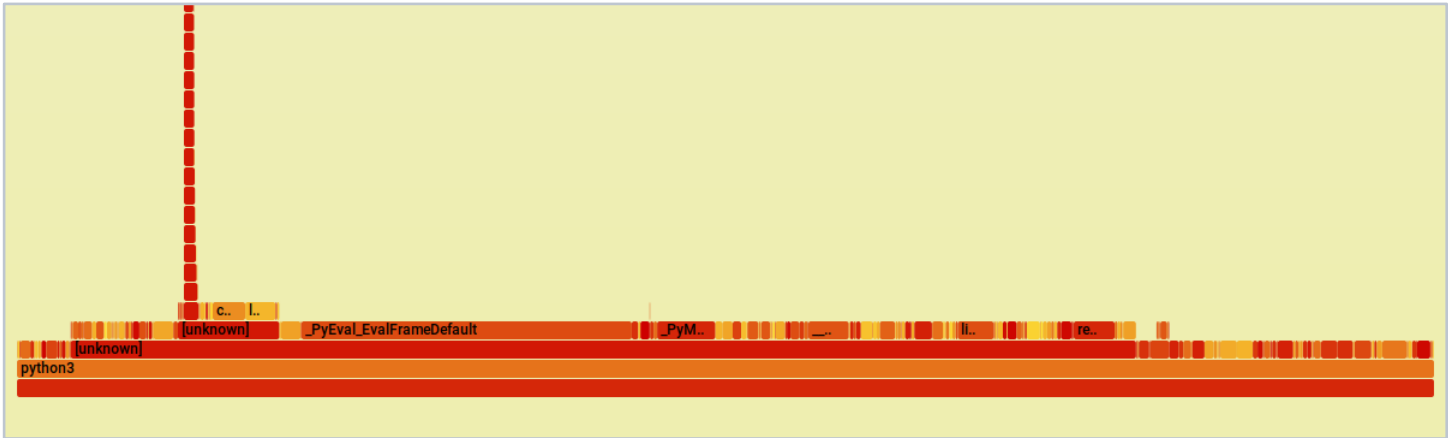
Huffman decode linearly scans code objects for every symbol

Bit masks are reconstructed repeatedly

Move-to-front rebuilds lists through slicing and concatenation

Pyflate baseline profile

47,455 CPU-clock samples; zero lost samples



- 23.31% CPython frame evaluation
- 4.09% allocator address checks
- 2.88% allocation-size reads
- 2.54% list deallocation
- 2.14% list slicing
- 1.14% list concatenation

Pyflate software optimization

Read compressed input once into a contiguous bytes buffer

Index bytes directly and cache masks for widths 0 through 64

Group canonical codes by valid bit length

Use $O(1)$ -average dictionary lookup instead of scanning code objects

Replace move-to-front slicing with in-place pop and insert

Keep block parsing, Huffman semantics, MTF/BWT/RLE, and output unchanged

Pyflate result: 1.43x faster

Baseline



1.121 s

Optimized



0.782 s

30.26% time reduction; required threshold: 7%

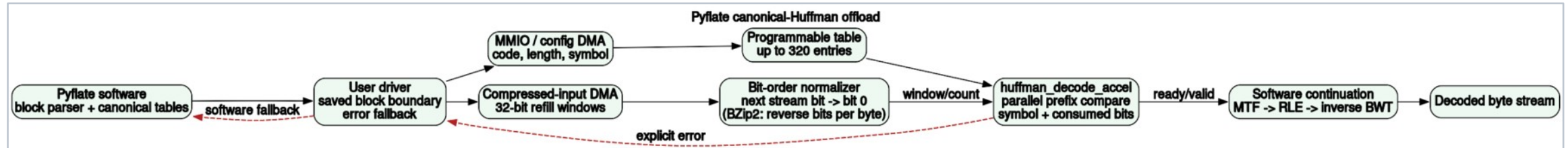
120 values each; CV below 1% for both runs

Total samples fall 27.2%

Absolute CPython evaluation time falls about 22%

List-slice sample count falls about 97%; concatenation is no longer significant

Pyflate hardware proposal



Programmable 1-20 bit canonical-Huffman lookup with explicit errors

32-bit normalized window; symbol and consumed-bit ready/valid output

BZip2 bytes are bit-reversed by the stream packer for transmission order

Production design banks all six BZip2 tables to avoid reprogramming

Conditional $P=0.50$, $S=10$ Amdahl estimate: 1.82x before DMA overhead

Hardware trade-offs and integration

Batch commands; never pay MMIO/PCIe latency per Python operation

Keep tables and intermediate state resident across many symbols/steps

Q16.16 reduces area/power but needs range checks and numerical validation

Parallel Huffman compare gives throughput but costs comparator area and power

Banked SRAM/prefix tables reduce area at the cost of extra latency

Every unsupported range, malformed table, or error returns to verified software

100-200 MHz targets are assumptions; no synthesis or power result is claimed

What the measurements taught us

N-body: interpreter removal exposes inverse-distance power as the new bottleneck

Pyflate: fewer method calls and temporary lists improve both time and locality

Algorithmic and data-layout changes should precede hardware

Amdahl's law and transfer overhead bound accelerator value

Debug-symbol perf profiles are useful, but unresolved VM/JIT frames must be disclosed

Correctness evidence and raw measurements are as important as headline speedup

Conclusion

N-body: 23.13x faster, 95.68% less time, identical validated state

Pyflate: 1.43x faster, 30.26% less time, byte-identical output

Both exceed the required 7% improvement

Both RTL designs pass self-checking simulation

Hardware proposals define interfaces, fallback, and realistic limitations

All scripts, raw JSON, perf reports, flame graphs, RTL, and prompts are reproducible